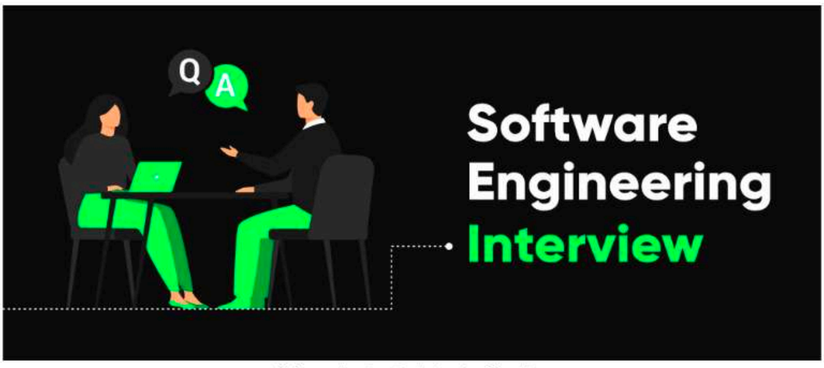
Top 50+ Software Engineering Interview Questions and Answers [2025]

Last Updated : 04 Apr, 2025

Software engineering is one of the most popular jobs in this technology-based world. The demand for creative software engineers is increasing as technology becomes important for businesses in various sectors.

Here is the list of **Top 50+ Software Engineering Interview Questions and Answers [2025]** that will guide you through both the basic and more advanced topics you're likely to face in interviews.



Software Engineering Interview Questions

Whether you're just starting out or already have some experience, these questions will give you a solid foundation to prepare and boost your confidence.

Table of Content

- Software Engineering Interview Questions for Beginner
- Software Engineering Interview Questions for Intermediate
- Software Engineering Interview Questions for Advance

Software Engineering Interview Questions for Beginner

Here are the Top Software Engineering Interview Questions for **Beginner**

1. What are the Characteristics of Software?

Top characteristics of software are:

- **Functionality:** It refers to the software performance compared to the purpose it was created for.
- **Reliability:** It is a characteristics of software that refers to its ability to perform what it was designed to do accurately and consistently over time.
- **Usability (User-friendly):** It refers to the extent to which the software can be used with ease. The amount of effort or time required to learn how to use the software.
- **Efficiency:** It refers to the ability of the software to use system resources in the most effective and efficient manner.
- **Flexibility:** It refers to how simple it is to improve and modify the software.
- **Maintainability:** It refers to how easily a software system can be modified to add feature, improve speed, or repair faults.
- **Portability:** It refers to how well the software can work on different platforms or situations without making major modifications.
- **Integrity:** It refers to how well the software maintains the accuracy and consistency of data throughout its cycle.

For more details please refer to the following article Characteristics of Software.

2. What are the Various Categories of Software?

The software is used extensively in several domains including hospitals, banks, schools, defense, finance, stock markets, and so on. It can be categorized into different types:

1. Based on Application

1. System Software- This type of software helps manage the hardware of your computer, like an operating system that controls how the computer works.
2. Application Software- These are the programs we use every day, such as word processors or music players, to perform specific tasks.
3. Web Application Software- These are programs we access via a web browser, like checking your email or managing your bank account online.

For more details please refer to the following article Characteristics of Software.

2. What are the Various Categories of Software?

The software is used extensively in several domains including hospitals, banks, schools, defense, finance, stock markets, and so on. It can be categorized into different types:

1. Based on Application

1. System Software- This type of software helps manage the hardware of your computer, like an operating system that controls how the computer works.
2. Application Software- These are the programs we use every day, such as word processors or music players, to perform specific tasks.
3. Web Applications Software- These are programs you access via a web browser, like checking your email or managing your bank account online.
4. **Embedded Software-**This software is built into devices like cars, home appliances, or factory machinery to help them function properly.
5. **Reservation Software**: Used to manage bookings and reservations, such as for hotels, flights, or restaurant tables.
6. **Business Software-** Designed to help businesses run smoothly, this software includes things like CRM (Customer Relationship Management) tools and ERP (Enterprise Resource Planning) systems.
7. **Artificial Intelligence Software-** These programs use machine learning to mimic human intelligence and perform tasks like recommendations or chatbots.
8. **Scientific Software-** Software used by researchers for tasks like simulations, analyzing data, or modeling experiments.

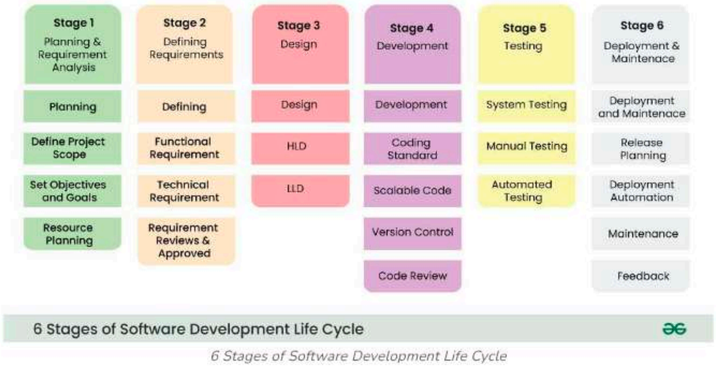
2. **Based on Copyright**

1. **Commercial Software**: This is software you pay for, usually with a license that limits how it can be used, shared, or changed.
2. **Shareware Software:** This software is available for free for a limited time or with limited features, with the hope that users will eventually pay for the full version.
3. **Freeware Software**: Software that's completely free to use, but the source code is usually not available for you to change or share.
4. **Public Domain Software**: This software is free for everyone to use, modify, and share without any copyright restrictions.

For more details please refer to the following article Classifications of Software.

3. Explain SDLC and its Phases?

SDLC stands for **Software Development Life Cycle**. It is a process followed for software building within a software organization. SDLC consists of a precise plan that describes how to develop, maintain, replace, and enhance specific software. The life cycle defines a method for improving the quality of software and the all-around development process.



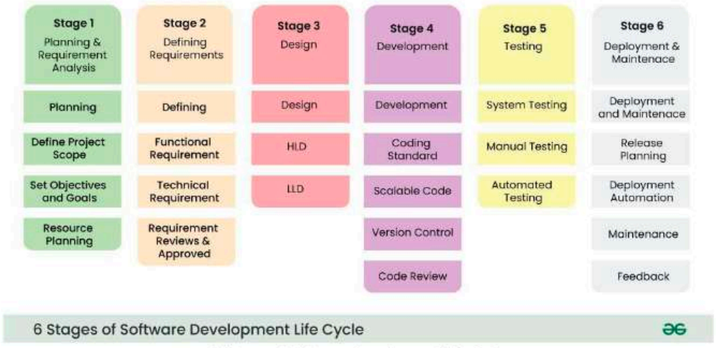
6 Stages of Software Development Life Cycle

Phases of SDLC: Following are the phases of SDLC:

1. **Planning and Requirement Analysis**: This is where we figure out the project's goals, understand what users need, and set clear expectations for what the software should do.
2. **Defining Requirements**: Here, we get into the specifics—laying out exactly what the software needs to do (functional) and how well it should do it (non-functional).
3. **Designing Architecture**: At this stage, we build a blueprint for the software, deciding on the overall structure and the technical details to make sure it meets the requirements.
4. **Developing Product**: This is where the coding happens, turning the design into a working product by writing the actual software.
5. Product Testing and Integration: Now, we test the software for bugs, check that everything

3. Explain SDLC and its Phases?

SDLC stands for **Software Development Life Cycle**. It is a process followed for software building within a software organization. SDLC consists of a precise plan that describes how to develop, maintain, replace, and enhance specific software. The life cycle defines a method for improving the quality of software and the all-around development process.



6 Stages of Software Development Life Cycle

Phases of SDLC: Following are the phases of SDLC:

1. **Planning and Requirement Analysis**: This is where we figure out the project's goals, understand what users need, and set clear expectations for what the software should do.
2. **Defining Requirements**: Here, we get into the specifics—laying out exactly what the software needs to do (functional) and how well it should do it (non-functional).
3. **Designing Architecture**: At this stage, we build a blueprint for the software, deciding on the overall structure and the technical details to make sure it meets the requirements.
4. **Developing Product**: This is where the coding happens, turning the design into a working product by writing the actual software.
5. **Product Testing and Integration**: Now, we test the software for bugs, check that everything works together smoothly, and make sure all the parts are integrated properly.
6. **Deployment and Maintenance of Products**: Finally, the software is deployed for use, and we continue to monitor and maintain it to fix any issues and keep it running smoothly over time.

For more details, please refer to the following article Software Development Life Cycle.

4. What are different SDLC Models Available?

Here are the models which is available for the SDLC:

1. Waterfall Model- This is a straightforward, step-by-step process where each phase must be finished before moving on to the next. It's ideal for projects with clear and fixed requirements from the start.
2. V-Model- Also called the Verification and Validation model, this approach focuses on testing each part of the development process as you go. For every phase of development, there's a corresponding testing phase.
3. Incremental Model- Instead of developing everything at once, this model breaks the software into smaller, manageable pieces or "increments." These pieces are developed and delivered one at a time, allowing users to get a working version early on.
4. RAD Model - This model focuses on getting the software up and running quickly through prototypes and constant user feedback. It works well for projects with clear user requirements and tight deadlines.
5. Iterative Model - In this approach, development happens in cycles. After each cycle, a version of the software is tested, feedback is gathered, and improvements are made in the next cycle.
6. Spiral Model - This model combines design and prototyping while focusing heavily on risk assessment. It involves repeating phases of planning, design, development, and testing with each loop, addressing any risks along the way.
7. Prototype model- Here, a working version of the product (prototype) is created early on. This allows users to interact with it and give feedback, which helps shape the final product.
8. Agile Model- Agile is all about flexibility. Development happens in short bursts, or sprints, with constant feedback from the customer. This model allows for changes at any stage, making it adaptable and responsive to evolving needs.

For more details, please refer to the following article Top Software Development Models (SDLC) Models.

5. What is the Waterfall Method and What are its Use Cases?

The waterfall model is a software development model used in the context of large, complex projects, typically in the field of information technology. It is characterized by a structured, sequential approach to project management and software development.

Phases of Waterfall Model:

1. Requirements Gathering and Analysis: This is the first step where you gather all the details

For more details, please refer to the following article Top Software Development Models (SDLC) Models.

5. What is the Waterfall Method and What are its Use Cases?

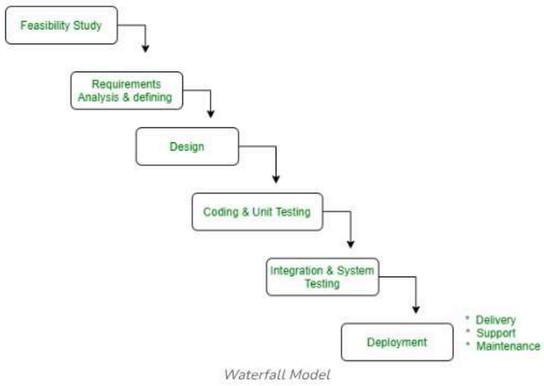
The waterfall model is a software development model used in the context of large, complex projects, typically in the field of information technology. It is characterized by a structured, sequential approach to project management and software development.

Phases of Waterfall Model:

1. **Requirements Gathering and Analysis**: This is the first step where you gather all the details about what the client needs and analyze them to understand exactly what the software should do.
2. **Design Phase**: Once you have a clear idea of the requirements, you move on to designing how the system will work. This is where you figure out the structure and how different parts of the software will fit together.
3. **Implementation and Unit Testing**: Now it's time to start coding. In this phase, developers write the actual software and test each small piece (or module) to make sure it works properly on its own.
4. **Integration and System Testing**: After all the individual modules are built and tested, they're put together to form the complete system. The whole system is tested to ensure everything works as expected when all the parts are connected.
5. **Deployment**: Once the software is tested and ready, it's launched and made available to users in the real world.
6. **Maintenance**: After the software is up and running, it enters the maintenance phase. This involves fixing any issues users find and making updates as needed to keep everything running smoothly.

Use Case of Waterfall Model

- Requirements are clear and fixed that may not change.
- There are no ambiguous requirements (no confusion).
- It is good to use this model when the technology is well understood.
- The project is short and cast is low.
- Risk is zero or minimum.



Waterfall Model

For more details, please refer to the following article Waterfall Model.

6. What is Black Box Testing?

The black box test (also known as the conducted test/ closed box test/ opaque box test) is software testing technique. In this technique, tester does not care about the internal knowledge or implementation details but rather focuses on validating the functionality based on the provided specifications or requirements. The name "black box" refers to the idea that the internal workings are hidden from the tester's view.

For more details, please refer to the following article Software Engineering - Black Box Testing.

7. What is White Box Testing?

White Box Testing is a method of analyzing the internal structure, data structures used, internal design, code structure, and behavior of software, as well as functions such as black-box testing. Also called glass-box test or clear box test or structural test.

For more details, please refer to the following article Software Engineering - White Box Testing.

8. Distinguish between Alpha and Beta Testing?

Following are the differences between Alpha and Beta Testing:

For more details, please refer to the following article Software Engineering - Black Box Testing.

7. What is White Box Testing?

White Box Testing is a method of analyzing the internal structure, data structures used, internal design, code structure, and behavior of software, as well as functions such as black-box testing. Also called glass-box test or clear box test or structural test.

For more details, please refer to the following article Software Engineering - White Box Testing.

8. Distinguish between Alpha and Beta Testing?

Following are the differences between Alpha and Beta Testing:

Alpha Testing	Beta Testing
Alpha testing involves both white box and black box testing.	Beta testing commonly uses black-box testing.
Alpha testing is performed by testers who are usually internal employees of the organization.	Beta testing is performed by clients who are not part of the organization.
Alpha testing is performed at the developer's site.	Beta testing is performed at the end-user, the of the product.
Reliability and security testing are not checked in alpha testing.	Reliability, security, and robustness are checked during beta testing.
Alpha testing ensures the quality of the product before forwarding it to beta testing.	Beta testing also concentrates on the quality of the product but collects the user's time-long input on the product and ensures that the product is ready for real-time users.
Alpha testing requires a testing environment or a lab.	Beta testing doesn't require a testing environment or lab.
Alpha testing may require a real-time long execution cycle.	Beta testing requires only a few weeks of execution.
Developers can immediately address the critical issues or fixes in alpha testing.	Most of the issues or feedback collected from the beta testing will be implemented in future versions of the product

For more details, please refer to the following article Alpha Testing and Beta Testing.

9. What is Debugging?

Debugging is the process of identifying and resolving errors, or bugs, in a software system. It is an important aspect of software engineering because bugs can cause a software system to malfunction, and can lead to poor performance or incorrect results. Debugging can be a time-consuming and complex task, but it is essential for ensuring that a software system is functioning correctly.

For more details, please refer to the following article What is Debugging?

10. What is a Feasibility Study?

The Feasibility Study in Software Engineering is a study that analyze whether a proposed software project is practical or not. It early detects the potential issues, analyzes technological possibilities, and determines the project's financial and operational viability. This decreases the chance of project failure that also save time and money.

For more details, please refer to the following article Types of Feasibility Study in Software Project Development article.

11. What is a Use Case Diagram?

A use case diagram is a behavior diagram and visualizes the observable interactions between actors and the system under development. The diagram consists of the system, the related use cases, and actors and relates these to each other

- System: What is being described?

Project Development article.

11. What is a Use Case Diagram?

A use case diagram is a behavior diagram and visualizes the observable interactions between actors and the system under development. The diagram consists of the system, the related use cases, and actors and relates these to each other:

- **System**: What is being described?
- **Actor**: Who is using the system?
- **Use Case**: What are the actors doing?

For more details, please refer to the following article use case diagram.

12. What is the difference between Verification and Validation?

Here are the difference between Verification and Validation

Verification	Validation
Verification is a static practice of verifying documents, design, code, black-box, and programs human-based.	Validation is a dynamic mechanism of validation and testing the actual product.
It does not involve executing the code.	It always involves executing the code.
It is human-based checking of documents and files.	It is computer-based execution of the program.
Verification uses methods like inspections, reviews, walkthroughs, and Desk-checking, etc.	Validation uses methods like black box (functional) testing, gray box testing, and white box (structural) testing, etc.
Verification is to check whether the software conforms to specifications.	Validation is to check whether the software meets the customer's expectations and requirements.
It can catch errors that validation cannot catch.	It can catch errors that verification cannot catch.
Target is requirements specification, application and software architecture, high level, complete design, and database design, etc.	Target is an actual product-a unit, a module, a bent of integrated modules, and an effective final product.
Verification is done by QA team to ensure that the software is as per the specifications in the SRS document.	Validation is carried out with the involvement of the testing team
It generally comes first done before validation.	It generally follows after verification.
It is low-level exercise.	It is a High-Level Exercise.

For more details, please refer to the following article Software Engineering - Verification and Validation.

13. What is a Baseline?

A baseline is a measurement that defines the completeness of a phase. After all activities associated with a particular phase are accomplished, the phase is complete and acts as a baseline for next phase.

For more details, please refer to the following article baseline.

Software Engineering Interview Questions for Intermediate

Here are the Top Software Engineering Interview Questions for **Intermediate:**

14. What is Cohesion and Coupling?

Cohesion indicates the relative functional capacity of the module. Aggregation modules need to interact less with other sections of other parts of the program to perform a single task. It can be said that only one coagulation module (ideally) needs to be run. Cohesion is a measurement of the functional strength of a module. A module with H cohesion and low coupling is functionally independent of other modules. Here, functional independence means that a cohesive module

For more details, please refer to the following article baseline.

Software Engineering Interview Questions for Intermediate

Here are the Top Software Engineering Interview Questions for **Intermediate:**

14. What is Cohesion and Coupling?

Cohesion indicates the relative functional capacity of the module. Aggregation modules need to interact less with other sections of other parts of the program to perform a single task. It can be said that only one coagulation module (ideally) needs to be run. Cohesion is a measurement of the functional strength of a module. A module with high cohesion and low coupling is functionally independent of other modules. Here, functional independence means that a cohesive module performs a single operation or function. The coupling means the overall association between the modules.

Coupling relies on the information delivered through the interface with the complexity of the interface between the modules in which the reference to the section or module was created. High coupling support Low coupling modules assume that there are virtually no other modules. It is exceptionally relevant when both modules exchange a lot of information. The level of coupling between two modules depends on the complexity of the interface.

For more details, please refer to the following article Coupling and cohesion.

15. What is the Agile software development model?

The agile SDLC model is a combination of iterative and incremental process models with a focus on process adaptability and customer satisfaction by rapid delivery of working software products. Agile Methods break the product into small incremental builds. Every iteration involves cross-functional teams working simultaneously on various areas like planning, requirements analysis, design, coding, unit testing, and acceptance testing.

Advantages:

- Customer satisfaction by rapid, continuous delivery of useful software.
- Customers, developers, and testers constantly interact with each other.
- Close, daily cooperation between business people and developers.
- Continuous attention to technical excellence and good design.
- Regular adaptation to changing circumstances.
- Even late changes in requirements are welcomed.

For more details, please refer to the following article Software Engineering - Agile Development Models.

16. What is the Difference Between Quality Assurance and Quality Control?

Quality Assurance (QA)	Quality Control (QC)
It focuses on providing assurance that the quality requested will be achieved.	It focuses on fulfilling the quality requested.
It is the technique of managing quality.	It is the technique to verify quality.
It does not include the execution of the program.	It always includes the execution of the program.
It is a managerial tool.	It is a corrective tool.
It is process-oriented.	It is product-oriented.
The aim of quality assurance is to prevent defects.	The aim of quality control is to identify and improve the defects.
It is a preventive technique.	It is a corrective technique.
It is a proactive measure.	It is a reactive measure.
It is responsible for the full software development life cycle.	It is responsible for the software testing life cycle.
Example: Verification	Example: Validation

- Even late changes in requirements are welcomed.

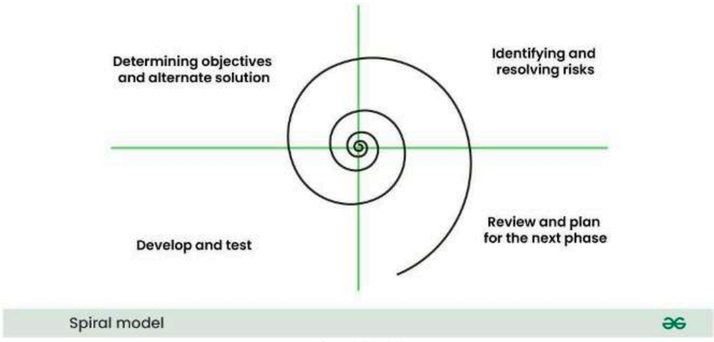
For more details, please refer to the following article Software Engineering - Agile Development Models.

16. What is the Difference Between Quality Assurance and Quality Control?

Quality Assurance (QA)	Quality Control (QC)
It focuses on providing assurance that the quality requested will be achieved.	It focuses on fulfilling the quality requested.
It is the technique of managing quality.	It is the technique to verify quality.
It does not include the execution of the program.	It always includes the execution of the program.
It is a managerial tool.	It is a corrective tool.
It is process-oriented.	It is product-oriented.
The aim of quality assurance is to prevent defects.	The aim of quality control is to identify and improve the defects.
It is a preventive technique.	It is a corrective technique.
It is a proactive measure.	It is a reactive measure.
It is responsible for the full software development life cycle.	It is responsible for the software testing life cycle.
Example: Verification	Example: Validation

17. What is the Spiral Model, and its Disadvantages?

The Spiral Model is a **Software Development Life Cycle (SDLC)** model that provides a systematic and iterative approach to software development. In its diagrammatic representation, looks like a spiral with many loops. The exact number of loops of the spiral is unknown and can vary from project to project. Each loop of the spiral is called a **phase** of the software development process.



Spiral Model

Following are the disadvantages of spiral model:

- Can be a costly model to use.
- Risk analysis requires highly specific expertise.
- The project's success is highly dependent on the risk analysis phase.
- Doesn't work well for smaller projects

For more details, please refer to the following article Software Engineering - Spiral Model.

18. What is the RAD Model, and its limitations?

IBM first proposed the Rapid Application Development or RAD Model in the 1980s. The RAD model is a type of incremental process model in which there is a concise development cycle. The RAD model is used when the requirements are fully understood and the component-based construction approach is adopted.

Following are the limitations of RAD Model:

- For large but scalable projects RAD requires sufficient human resources.
- Projects fail if developers and customers are not committed in a much-shortened time frame.
- Problematic if a system cannot be modularized

For more details, please refer to the following article Software Engineering - Spiral Model.

18. What is the RAD Model, and its limitations?

IBM first proposed the Rapid Application Development or RAD Model in the 1980s. The RAD model is a type of incremental process model in which there is a concise development cycle. The RAD model is used when the requirements are fully understood and the component-based construction approach is adopted.

Following are the limitations of RAD Model:

- For large but scalable projects RAD requires sufficient human resources.
- Projects fail if developers and customers are not committed in a much-shortened time frame.
- Problematic if a system cannot be modularized

For more details, please refer to the following article Software Engineering - Rapid Application Development Model (RAD).

19. What is Regression Testing?

Regression testing is defined as a type of software testing that is used to confirm that recent changes to the program or code have not adversely affected existing functionality. Regression testing is just a selection of all or part of the test cases that have been run. These test cases are rerun to ensure that the existing functions work correctly. This test is performed to ensure that new code changes do not have side effects on existing functions. Ensures that after the last code changes are completed, the above code is still valid.

*For more details, please refer to the following article **regression testing**.*

20. What are CASE Tools?

CASE stands for Computer-Aided Software Engineering. CASE tools are a set of automated software application programs, which are used to support, accelerate and smoothen the SDLC activities. It is a software package that helps with the design and deployment of information systems. It can record a database design and be quite useful in ensuring design consistency.

21. What is Physical and Logical DFD (Data Flow Diagram) ?

Physical DFD and Logical DFD both are the types of DFD (Data Flow Diagram) used to represent how data flows within a system.

Physical DFD focuses on how the system is implemented. The next diagram to draw after creating a logical DFD is physical DFD. It explains the best method to implement the business activities of the system. Moreover, it involves the physical implementation of devices and files required for the business processes. In other words, physical DFD contains the implantation-related details such as hardware, people, and other external components required to run the business processes.

Logical data flow diagram mainly focuses on the system process. It illustrates how data flows in the system. In the Logical Data Flow Diagram (DFD), we focus on the high-level processes and data flow without delving into the specific implementation details. Logical DFD is used in various organizations for the smooth running of system. Like in a Banking software system, it is used to describe how data is moved from one entity to another.

22. What is Software Re-engineering?

Software re-engineering is the process of scanning, modifying, and reconfiguring a system in a new way. The principle of reengineering applied to the software development process is called software reengineering. It has a positive impact on software cost, quality, customer service, and shipping speed. Software reengineering improves software to create it more efficiently and effectively.

For more details please refer to What Is Software Re-Engineering?

23. What is Reverse Engineering?

Software Reverse Engineering is a process of recovering the design, requirement specifications, and functions of a product from an analysis of its code. It builds a program database and generates information from this. The purpose of reverse engineering is to facilitate maintenance work by improving the understandability of a system and producing the necessary documents for a legacy system.

Reverse Engineering Goals:

- Cope with Complexity.
- Recover lost information.
- Detect side effects.
- Synthesize higher abstraction.
- Facilitate Reuse

For more details please refer to What Is Software Re-Engineering?

23. What is Reverse Engineering?

Software Reverse Engineering is a process of recovering the design, requirement specifications, and functions of a product from an analysis of its code. It builds a program database and generates information from this. The purpose of reverse engineering is to facilitate maintenance work by improving the understandability of a system and producing the necessary documents for a legacy system.

Reverse Engineering Goals:

- Cope with Complexity.
- Recover lost information.
- Detect side effects.
- Synthesize higher abstraction.
- Facilitate Reuse.

For more details, please refer to the following article Software Engineering - Reverse Engineering.

24. What are Software Project Estimation Techniques Available?

There are some software project estimation techniques available:

- PERT
- WBS
- Delphi method
- User case point

*For more details, please refer to the following article **Software Project Estimation Techniques**.*

25. How to Measure the Complexity of Software?

To measure the complexity of software there are some methods in software engineering:

- Line of codes
- Cyclomatic complexity
- Class coupling
- Depth of inheritance

For more details, please refer to the following article complexity of software.

26. Mentions Some Software Analysis and Design Tools?

Following are some software analysis and design tools:

- Data Flow Diagrams
- Structured Charts
- Structured English
- Data Dictionary
- Hierarchical Input Process Output diagrams
- Entity Relationship Diagrams and Decision tables

27. What is the Name of Various CASE Tools?

- Requirement Analysis Tool
- Structure Analysis Tool
- Software Design Tool
- Code Generation Tool
- Test Case Generation Tool
- Document Production Tool
- Reverse Engineering Tool

For more details, please refer to the following article Computer-Aided Software Engineering(CASE).

28. What is SRS?

Software Requirement Specification (SRS) Format is a complete specification and description of requirements of the software that needs to be fulfilled for successful development of software system. These requirements can be functional as well as non-requirements depending upon the type of requirement. The interaction between different customers and contractors is done because it is necessary to fully understand the needs of

Engineering(CASE).

28. What is SRS?

Software Requirement Specification (SRS) Format is a complete specification and description of requirements of the software that needs to be fulfilled for successful development of software system. These requirements can be functional as well as non-requirements depending upon the type of requirement. The interaction between different customers and contractors is done because it is necessary to fully understand the needs of customers.

For more details please refer software requirement specification format article.

29. What is level-0 DFD?

The highest abstraction level is called Level 0 of DFD. It is also called context-level DFD. It portrays the entire information system as one diagram.

For more details, please refer to the following article DFD.

30. What is a Function Point?

Function point metrics provide a standardized method for measuring the various functions of a software application. Function point metrics, measure functionality from the user's point of view, that is, on the basis of what the user requests and receives in return.

*For more details, please refer to the following article **function point.***

31. What is the formula to Calculate the Cyclomatic Complexity?

The formula to calculate the cyclomatic complexity of a program is:

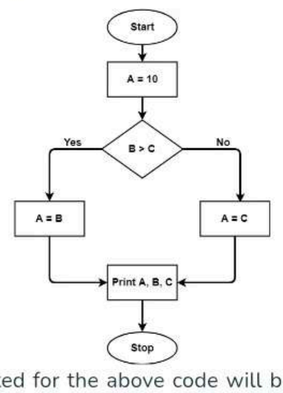
$$c = e - n + 2p \qquad \text{where,}$$

1. *e = number of edges*
2. *n = number of vertices*
3. *p = predicates*

Example:

```
A = 10
   IF B > C THEN
      A = B
   ELSE
      A = C
   ENDIF
Print A
Print B
Print C
```

Control Flow Graph of the above code:



The cyclomatic complexity calculated for the above code will be from the control flow graph. The graph shows seven shapes(nodes), and seven lines(edges), hence cyclomatic complexity is 7-7+2 = 2.

32. What is the Cyclomatic Complexity of a Module that has 17 Edges and 13 Nodes?

The Cyclomatic complexity of a module that has seventeen edges and thirteen nodes = E – N + 2

E = Number of edges, N = Number of nodes
Cyclomatic complexity = 17 – 13 + 2 = 6

33. What is COCOMO Model?

A COCOMO model stands for Constructive Cost Model. As with all estimation models, it requires

graph shows seven shapes(nodes), and seven lines(edges), hence cyclomatic complexity is 7-7+2 = 2.

32. What is the Cyclomatic Complexity of a Module that has 17 Edges and 13 Nodes?

The Cyclomatic complexity of a module that has seventeen edges and thirteen nodes = E – N + 2

E = Number of edges, N = Number of nodes
Cyclomatic complexity = 17 – 13 + 2 = 6

33. What is COCOMO Model?

A COCOMO model stands for Constructive Cost Model. As with all estimation models, it requires sizing information and accepts it in three forms:

- Object points
- Function points
- Lines of source code

For more details, please refer to the following article Software Engineering - COCOMO Model.

34. Define an Estimation of Software Development Effort for Organic Software in the basic COCOMO Model?

Estimation of software development effort for organic software in the basic COCOMO model is defined as

```
Organic: Effort = 2.4(KLOC) 1.05 PM
```

Software Engineering Interview Questions for Advance

Here are the Top Software Engineering Interview Questions for **Advance:**

35. What Activities Come Under the Umbrella Activities?

The activities of the software engineering process framework are complemented by a variety of higher-level activities. Umbrella activities typically apply to the entire software project and help the software team manage and control progress, quality, changes, and risks. Common top activities include Software Project Tracking and Control Risk Management, Software Quality Assurance Technical Review Measurement Software Configuration Management Reusability Management Work Product Preparation and Production, etc.

For more details, please refer to the following article Umbrella activities in Software Engineering.

36. Which SDLC Model is the Best?

The selection of the best SDLC model is a strategic decision that requires a thorough understanding of the project's requirements, constraints, and goals. While each model has its strengths and weaknesses, the key is to align the chosen model with the specific characteristics of the project. Being flexible, adaptable, and communicating well are crucial in dealing with the complexities of making software and making sure the final product is good. In the end, the best way to develop software is the one that suits the project's needs and situation the most.

For more details, please refer to the following article Which SDLC Model is Best and Why ?

37. What is the Black Hole Concept in DFD?

A block hole concept in the data flow diagram can be defined as "A processing step may have input flows but no output flows". In a black hole, data can only store inbound flows.

38. Which of the Testing is Used for Fault Simulation?

With increased expectations for software component quality and the complexity of components, software developers are expected to perform effective testing. In today's scenario, mutation testing has been used as a fault injection technique to measure test adequacy. Mutation Testing adopts "fault simulation mode".

39. Which Model is Used to Check Software Reliability?

A Rayleigh model is used to check software reliability. The Rayleigh model is a parametric model in the sense that it is based on a specific statistical distribution. When the parameters of the statistical distribution are estimated based on the data from a software project, projections about the defect rate of the project can be made based on the

has been used as a fault injection technique to measure test adequacy. Mutation Testing adopts "fault simulation mode".

39. Which Model is Used to Check Software Reliability?

A Rayleigh model is used to check software reliability. The Rayleigh model is a parametric model in the sense that it is based on a specific statistical distribution. When the parameters of the statistical distribution are estimated based on the data from a software project, projections about the defect rate of the project can be made based on the model.

40. What is the Difference between Risk and Uncertainty?

- Risk is able to be measured while uncertainty is not able to be measured.
- Risk can be calculated while uncertainty can never be counted.
- You are capable of make earlier plans in order to avoid risk. It is impossible to make prior plans for the uncertainty.
- Certain sorts of empirical observations can help to understand the risk but on the other hand, the uncertainty can never be based on empirical observations.
- After making efforts, the risk is able to be converted into certainty. On the contrary, you can't convert uncertainty into certainty.
- After making an estimate of the risk factor, a decision can be made but as the calculation of the uncertainty is not possible, hence no decision can be made.

41. What is CMM?

To determine an organization's current state of process maturity, the SEI uses an assessment that results in a five-point grading scheme. The grading scheme determines compliance with a capability maturity model (CMM) that defines key activities required at different levels of process maturity. The SEI approach provides a measure of the global effectiveness of a company's software engineering practices and establishes five process maturity levels that are defined in the following manner:

- Level 1: Initial
- Level 2: Repeatable
- Level 3: Defined
- Level 4: Managed
- Level 5: Optimizing

For more details, please refer to the following article CMM.

43. A software does not wear out in the traditional sense of the term, but the software does tend to deteriorate as it evolves, why?

The software does not wear out in the traditional sense of the term, but the software does tend to deteriorate as it evolves because Multiple change requests introduce errors in component interactions. Unlike hardware, software doesn't physically wear out. However, it tends to deteriorate as it evolves because every time new features or updates are added, there's a chance of introducing new bugs or compatibility issues. Over time, multiple changes can cause different parts of the software to interact in unexpected ways, making it harder to maintain. If these issues aren't managed properly, the software becomes more complex and prone to failure, which affects its performance and reliability.

44. What are the elements to be considered in the System Model Construction?

The type and size of the software, the experience of use for reference to predecessors, difficulty level to obtain users' needs, development techniques and tools, the situation of the development team, development risks, the software development methods should be kept in mind. It is an important prerequisite to ensure the success of software development that designing a reasonable and suitable software development plan.

45. Define Adaptive Maintenance?

Adaptive maintenance defines as modifications and updating when the customers need the product to run on new platforms, on new operating systems, or when they need the product to interface with new hardware and software.

46. Define the term WBS?

The full form of WBS is Work Breakdown Structure. Its **Work Breakdown Structure** includes dividing a large and complex project into simpler, manageable, and independent tasks. For constructing a work breakdown structure, each node is recursively decomposed into smaller sub-activities, until at the leaf level, the activities become undividable and independent. A WBS works on a top-down approach.

with new hardware and software.

46. Define the term WBS?

The full form of WBS is Work Breakdown Structure. Its **Work Breakdown Structure** includes dividing a large and complex project into simpler, manageable, and independent tasks. For constructing a work breakdown structure, each node is recursively decomposed into smaller sub-activities, until at the leaf level, the activities become undividable and independent. A WBS works on a top-down approach.

For more detail please refer Work breakdown structure article.

47. How to Find the Size of a Software Product?

Estimation of the size of the software is an essential part of Software Project Management. It helps the project manager to further predict the effort and time which will be needed to build the project. Various measures are used in project size estimation. Some of these are:

- Lines of Code
- Number of entities in ER diagram
- Total number of processes in detailed data flow diagram
- Function points

48. What is Concurrency, and How to Achieve it ?

Concurrency refers to a system's ability to execute numerous tasks or processes at the same time, ostensibly concurrently. It is a wider concept that includes the idea of completing many jobs in concurrent intervals. In a concurrent system, tasks can begin, run, and finish in overlapping time frames, increasing overall system efficiency and responsiveness. There are many programming language available that support concurrency using unthreading example Java, C++ etc.

49. Why is Modularization important in Software Engineering?

Modular programming makes your code easier to read by dividing it into functions that only deal with one part of the overall functionality. When compared to monolithic code, it can make your files significantly smaller and easier to read.

50. Which Process Model Removes Defects before Software get into trouble?

Clean room software engineering is a software development approach to producing quality software. In clean room software engineering, an efficient and good quality software product is delivered to the client as QA (Quality Assurance) is performed each and every phase of software development.

51. Difference between an EXE and DLL?

DLL refers to Dynamic Link Library, and EXE is an executable. An EXE assembly can execute in its own address space, whereas a DLL cannot. It does not have its own address space, therefore it must run within a host and requires a consumer to invoke it.

Conclusion

Preparing for a **Software Engineering Interview** means having a solid foundation of both the basics and more advanced topics, like **SDLC models**, **testing methods**, and **design principles**. By mastering these areas, practicing problem-solving, and staying up-to-date with new technologies, You will be ready to clear any interview confidently with the well knowledge.

- Software Engineering Interview Questions for Freshers
- Software Engineering Interview Questions for Experienced

Software Engineering Interview Questions for Freshers

1. What is baseline in Software Development?

A baseline is a software development milestone and reference point marked by the completion or delivery of one or more software deliverables. The main objective of the baseline is to decrease and regulate vulnerability, or project weaknesses that can easily damage the project and lead to uncontrollable changes.

2. What do you mean by Software Re-engineering?

The process of updating software is known as software reengineering. This procedure entails adding new features and functionalities to the software in order to make it better and more efficient.

3. What are Verification and Validation?

- **Verification:** The process of ensuring that software accomplishes its objectives without defects is known as verification. It's the procedure for determining whether the product being developed is correct or not. It determines whether the resulting product meets our specifications. It is mainly focused on functionality.
- **Validation:** Validation is the process of determining whether a software product meets the required standards, or in other words, whether it meets the product's quality criteria. It is the process of verifying product validation or ensuring that the product we are building is correct. Validation focuses on the quality of the software.

4. What are CASE tools?

CASE tools are a collection of software application programs that automate SDLC tasks. Analysis tools, Design tools, Project management tools, Database Management tools, and Documentation tools are a few of the CASE tools available to simplify various stages of the Software Development Life Cycle.

5. What is SRS?

SRS is a formal report that serves as a representation of software that allows customers to assess whether it meets their needs. It is a list of requirements for a certain software product, program, or set of apps that execute specific tasks in a specific environment. It also includes user needs for a system, as well as precise system requirements specifications. Depending on who is writing it, it fulfils a variety of purposes.

6. What are the various categories of software?

Software products are mainly categorized into:

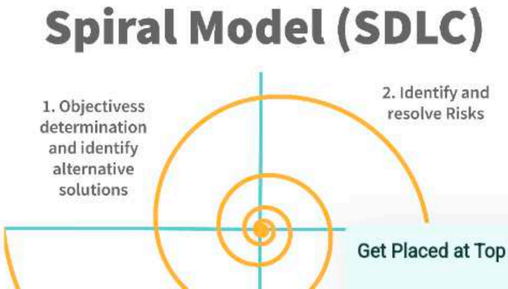
- **System software:** Softwares like operating systems, compilers, drivers, etc. fall into this category.
- **Networking and web development software:** Computer networking software offers the necessary functionality for computers to communicate with one another and with data storage facilities.
- **Embedded Software:** Software used in instrumentation and control applications such as washing machines, satellites, microwaves, TVs, etc.
- **Artificial Intelligence Software:** Expert systems, decision support systems, pattern recognition software, artificial neural networks, and other types of software are included in this category.
- **Scientific software:** These support a scientific or engineering user's requirements for performing enterprise-specific tasks. Examples include MATLAB, AUTOCAD, etc.

7. What are the drawbacks of the spiral model?

The spiral model is a hybrid of the iterative development process and the waterfall model, with a focus on risk analysis. In the SDLC Spiral model, the development process begins with a limited set of requirements and progresses through each development phase. Until the application is ready for production, the software engineering team adds functionality for the increased requirement in ever-increasing spirals.

Drawbacks of the spiral model are:

- It's significantly more complicated than other SDLC models. The procedure is intricate.
- Due to its high cost, it is not recommended for small projects.
- Risk Analysis is overly reliant, and it necessitates a high level of skill.
- Time estimation is challenging
- The spiral could continue endlessly.



- Due to its high cost, it is not recommended for small projects.
- Risk Analysis is overly reliant, and it necessitates a high level of skill.
- Time estimation is challenging
- The spiral could continue endlessly.

8. What is Software prototyping and POC?

A software prototype is a working model with limited functionality. The prototype may or may not contain the exact logic used in the final software program, and therefore is an additional work that should be considered in the calculation. Users can review developer proposals and try them out before they are implemented through prototyping. It also helps in comprehending user-specific details that may have been missed by the developer during product development.

POC (Proof of Concept) is a method used by organizations to validate an idea or concept's practicality. The stage exists prior to the start of the software development process. On the basis of technical capability and business model, a mini project is built to see if a concept can be executed.

9. What are the merits of the incremental model?

- It can deliver iteration faster, in the first iteration itself.
- Development takes place in parallel to each other.
- We can reduce the first delivery cost by using this method.
- The user or client can provide feedback at each level and unexpected changes in the requirement can be avoided.
- Risks can be identified and managed on a module-by-module basis.

10. What is Software scope?

The scope of a software project is a well-defined boundary that incorporates all the activities involved in developing and delivering a software product. The scope defines what the product will and will not do, as well as what the final product will and will not contain. All capabilities and objects to be delivered as part of the software are explicitly defined in the software scope.

11. What is the waterfall method and what are its use cases?

The waterfall is the easiest and most straightforward SDLC approach in software development. In this approach, the development process is linear, and each step is finished one by one. As the name implies, development progresses downwards, much like a waterfall. The software has to cover the following phases in a waterfall model:

- Requirements
- Design
- Implementation
- Testing and integration
- Deployment
- Maintenance

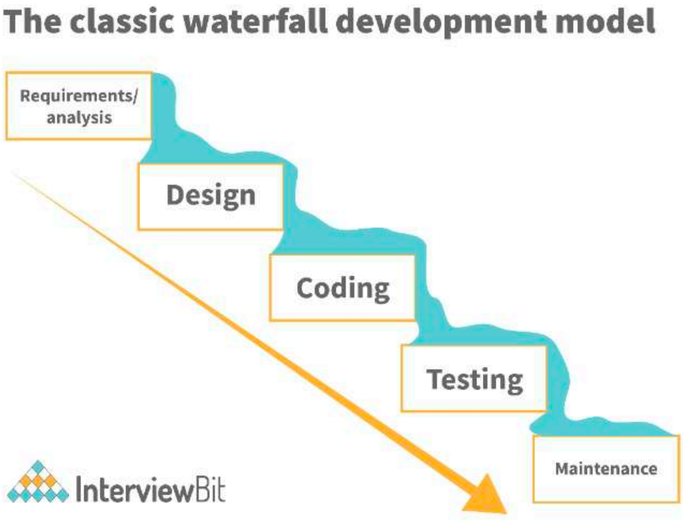
Use cases:

- When requirements are well-defined and unchangeable.
- There are no ambiguous requirements or conditions.
- When the technology is well understood
- The project is brief, and the cast is small.
- The risk is negligible.

- Requirements
- Design
- Implementation
- Testing and integration
- Deployment
- Maintenance

Use cases:

- When requirements are well-defined and unchangeable.
- There are no ambiguous requirements or conditions.
- When the technology is well understood
- The project is brief, and the cast is small.
- The risk is negligible.



12. What does a software product manager do?

A software product manager leads and manages the software product management department. They are in charge of the software product's specialization, goals, structure, and expectations. They also head the planning, backlog grooming, stakeholder management, and providing roadmap necessary to create the best software.

13. What is Debugging?

Debugging is the process of finding a software bug, in the context of software engineering. To put it another way, it refers to the process of finding, evaluating, and correcting problems. Debugging is essential once the software fails to run properly.

14. Which SDLC model is the best?

According to the annual State of Agile report, Agile is the best SDLC methodology and also one of the most widely used SDLC in the IT industry. The reason is that it is a hybrid of incremental and iterative approaches and works well in a flexible environment. That being said, select the model that suits your requirements.

15. What are different SDLC models available?

- Waterfall model
- Spiral model
- Incremental model
- Agile Model
- Big bang model
- Iterative model

16. Describe the Software Development Process in Brief.

The Software Development Life Cycle (SDLC) is a number of fundamental phases that teams must follow in order to produce and deliver high-quality software. Software typically goes through the following phases:

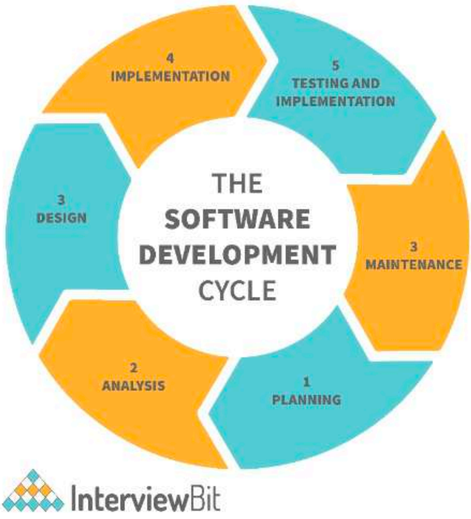
- **Requirements Gathering:** The team identifies, collects, and defines core issues, requirements, requests, and customer expectations related to the software application or service during this stage of the project. Generating software specifications, creating a thorough strategy, documentation, issue tracking, and project or product planning, including allocating the resources, are some tasks done during this phase.
- **Software Design:** The team makes software design decisions regarding the architecture and make of the software solution throughout this design phase of development.
- **Software Development:** Teams develop software solutions based on the design decisions made during earlier stages of the project
- **Testing and Integration:** Software is packaged and tested to ensure quality. Quality assurance, often known as testing, ensures that the solutions deployed fulfil the specified quality and performance criteria.
- **Deployment:** The software is installed in a production setting. The gathered, designed, built, and tested work is shared with the software service's customers and users.
- **Operation and Maintenance:** The software is installed in a production setting. The work is shared with the software service's customers and users.

- Agile Model
- Big bang model
- Iterative model

16. Describe the Software Development Process in Brief.

The Software Development Life Cycle (SDLC) is a number of fundamental phases that teams must follow in order to produce and deliver high-quality software. Software typically goes through the following phases:

- **Requirements Gathering:** The team identifies, collects, and defines core issues, requirements, requests, and customer expectations related to the software application or service during this stage of the project. Generating software specifications, creating a thorough strategy, documentation, issue tracking, and project or product planning, including allocating the resources, are some tasks done during this phase.
- **Software Design:** The team makes software design decisions regarding the architecture and make of the software solution throughout this design phase of development.
- **Software Development:** Teams develop software solutions based on the design decisions made during earlier stages of the project
- **Testing and Integration:** Software is packaged and tested to ensure quality. Quality assurance, often known as testing, ensures that the solutions deployed fulfil the specified quality and performance criteria.
- **Deployment:** The software is installed in a production setting. The gathered, designed, built, and tested work is shared with the software service's customers and users.
- **Operation and Maintenance:** The software is installed in a production setting. The work is shared with the software service's customers and users.



17. What is the main difference between a computer program and computer software?

The key difference between software is a collection of several programs used to complete tasks, whereas a program is a set of instructions expressed in a programming language. A program can be software, but software the vice versa is not true.

18. What is a framework?

A framework is a well-known method of developing and deploying software. It is a set of tools that allows developing software by providing information on how to make it on an abstract level, rather than giving exact details. The Software Process Framework is the basis of the entire software development process. The umbrella activities are also included in the software process structure.

19. What are the characteristics of software?

There are six major characteristics of software:

- **Functionality**: The things that software is intended to do are called functionality. For example, a calculator's functionality is to perform mathematical operations.
- **Efficiency:** It is the ability of the software to use the provided resources in the best way possible. Increasing the efficiency of software increases resource utilization and reduces cost.
- **Reliability:** Reliability is the probability of failure-free operational software in an environment. It is an important characteristic of software.
- **Usability:** It refers to the user's experience while using the software. Usability determines the satisfaction of the user.
- **Maintainability:** The ease with which you can repair, improve, and comprehend software code is referred to as maintainability. After the customer receives the product, a phase in the software development cycle called software maintenance begins.
- **Portability:** It refers to the ease with which the software product can be transferred from one environment to another.

Apart from the above-mentioned characteristics, the software also has the following characteristics:

- Software is engineered, it is not developed or manufactured like hardware. Development is an aspect of the hardware manufacturing process. Manufacturing does not exist in the case of software.
- The software doesn't wear out.
- The software is custom-built.

Software Engineering Interview Questions for Experienced

20. What is the difference between Quality Assurance and Quality control?

Quality Assurance	Quality Control
Quality Assurance focuses on ensuring that the	Quality contro

- The software doesn't wear out.
- The software is custom-built.

Software Engineering Interview Questions for Experienced

20. What is the difference between Quality Assurance and Quality control?

Quality Assurance	Quality Control
Quality Assurance focuses on assuring that the end product (software) will be of the requested quality.	Quality control focuses on controlling the processes, methods, or techniques used in the development of software so that the quality requested is fulfilled.
It is a preventive measure.	It is a corrective measure.
It applies to the full software development life cycle.	It is applied in the testing phase.

Conclusion

Software engineering is a lucrative job, and it requires hard work and dedication to become one. Becoming aware of questions asked in interviews can really help a lot. We covered software engineering questions that can help you crack that interview. The above list of relevant questions can only be a guideline. We cannot predict the exact problem that may pop up during the interview, but we hope that the general architecture and design knowledge gained from them would be helpful for you.

- Software Engineer / Developer Salary in India
- Facebook Software Engineer Salary
- Apple Software Engineer Salary
- Amazon Software Engineer Salary
- Software Engineer Salary in New York
- Software Engineer Salary in Texas
- Software Engineering Books
- Software Developer Vs Software Engineer
- Software Engineer MCQs
- Agile Interview Questions
- SDLC vs STLC: Full Difference
- Top Software Engineering Projects
- Latest Software Engineering Books
- Software Engineer Resume – Full Guide and Example

21. What are functional and non-functional requirements?

Functional Requirements	Non-functional Requirements
These are the needs that the end-user specifies as essential features that the system should provide.	These are the quality requirements that the system must meet in order to fulfil the project contract.
The user specifies the functional requirements.	Technical individuals, such as architects, technical leaders, and software engineers, specify non-functional requirements.
Functional Requirements are mandatory. For example, the client might want certain mandatory changes in UI, like dark mode.	Non-functional requirements are not Mandatory. For example, the requirement to enhance readability is non-functional.

22. What is Software Configuration Management?

When a piece of software is created, there is always room for improvement. To modify or improve an existing solution or to establish a new solution for a problem, changes may be required. Changes to the existing system should be examined before being implemented, recorded before being implemented, documented with details of before and after, and controlled in a way that improves quality and reduces error. This is where System Configuration Management is required.

During the Software Development Life Cycle, Software Configuration Management (SCM) is a technique for systematically managing, organizing, and controlling changes in documents, codes, and other entities. The main goal is to enhance production while making as few mistakes as possible.

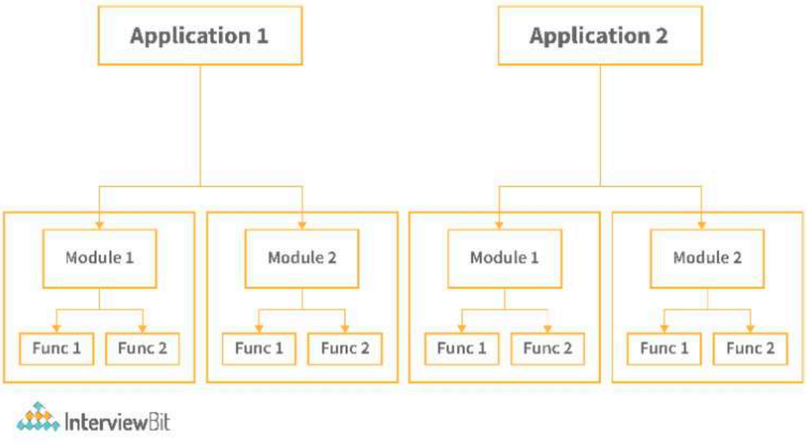
23. Explain the concept of modularization.

Modularization is breaking down a program's functionality into separate, independent modules, each of which includes just the information needed to carry out one part of the intended capability. In simple terms, it is the practice of dividing the program into smaller modules so that we can deal with them separately. We can simply add independent and smaller modules to a program using modularization without being hampered by the complexity of the program's other functionalities. Modularization is based on the notion of designing applications that are easier to develop and maintain, self-contained components. In monolithic design, on the other hand, there's always the risk of a simple change knocking the entire application down. The final step would be to combine these independent modules.

During the Software Development Life Cycle, Software Configuration Management (SCM) is a technique for systematically managing, organizing, and controlling changes in documents, codes, and other entities. The main goal is to enhance production while making as few mistakes as possible.

23. Explain the concept of modularization.

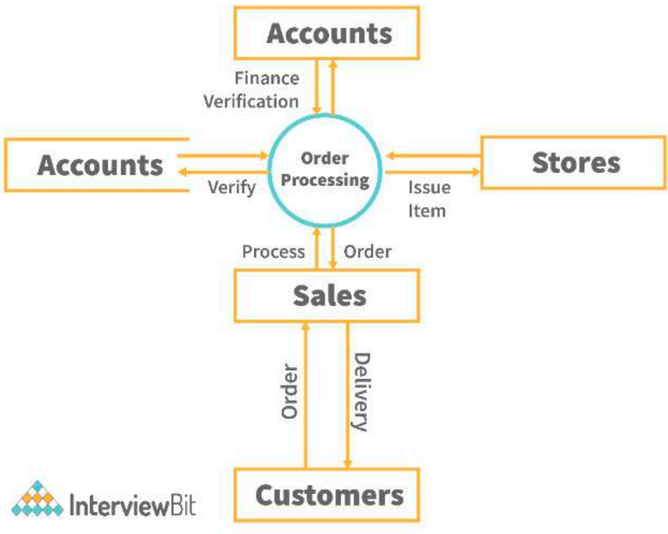
Modularization is breaking down a program's functionality into separate, independent modules, each of which includes just the information needed to carry out one part of the intended capability. In simple terms, it is the practice of dividing the program into smaller modules so that we can deal with them separately. We can simply add independent and smaller modules to a program using modularization without being hampered by the complexity of the program's other functionalities. Modularization is based on the notion of designing applications that are easier to develop and maintain, self-contained components. In monolithic design, on the other hand, there's always the risk of a simple change knocking the entire application down. The final step would be to combine these independent modules.



In the above diagram, both the applications have been divided into smaller modules. These modules can then be dealt with separately.

24. What is Data Flow Diagram?

A **Data Flow Diagram** (DFD) shows the flow of information flows through a system. It shows data inputs, outputs, storage sites, and paths between each destination using symbols such as rectangles, circles, and arrows, as well as short text labels. Data flowcharts can range from simple to in-depth DFDs that go deeper into how data is processed. They can be used to evaluate a current system or to create a new system. A DFD can effortlessly express things that are difficult to describe in words, and it can be used by both technical and non-technical audiences.



25. What is the difference between cohesion and coupling?

Cohesion	Coupling
Cohesion refers to the relationship within modules.	Coupling refers to the relationship between modules.
Increasing cohesion is good for the software.	Coupling should be avoided.
Modules focus on a particular thing in cohesion.	Modules are coupled to one another through coupling.
Example: A function that checks file permission and then opens it, or a function to decrypt messages.	**Example:** Two models sharing data with each other.

26. What are Software Metrics?

A software metric is a quantitative measure of program properties. Software metrics can be used for a range of things, such as analyzing software performance, planning, estimating productivity, and so on. Load testing, stress testing, average failure rate, code complexities, lines of code, etc. are some software metrics. The benefits o

- It reduces cost

Increasing cohesion is good for the software.	Coupling should be avoided.
Modules focus on a particular thing in cohesion.	Modules are coupled to one another through coupling.
Example: A function that checks file permission and then opens it, or a function to decrypt messages.	**Example:** Two models sharing data with each other.

26. What are Software Metrics?

A software metric is a quantitative measure of program properties. Software metrics can be used for a range of things, such as analyzing software performance, planning, estimating productivity, and so on. Load testing, stress testing, average failure rate, code complexities, lines of code, etc. are some software metrics. The benefits of software metrics are many, some of them being:

- It reduces cost.
- It increases ROI (return on investment).
- Reduces workload.
- Highlights areas for improvement.

27. What is Concurrency?

In software engineering, concurrency refers to a set of techniques and mechanisms that allow the software to do many tasks at the same time. Concurrency can be achieved by using languages like C++ or Java because these languages support the concept of thread. New hardware and software features are required to achieve concurrency.

28. Define black box testing and white box testing?

- **Black box testing** is a type of high-level testing in which the primary goal is to evaluate functionalities from a behavioural standpoint. In black-box testing, the tester does not test the code; instead, they utilize the program to see if it works as expected.
- When you have insight into the code or broad information about the architecture of the software in question, you can perform **white box testing**, also known as **clear box testing**. It falls under the category of low-level testing and is mostly concerned with integration and unit testing. White box testing requires programming expertise or at the very least a thorough grasp of the code that implements a particular functionality.

29. What is the feasibility study?

As the name implies, a feasibility study is a measurement of a software product in terms of how useful product development will be for the business from a practical standpoint. Feasibility studies are conducted for a variety of reasons, including determining whether a software product is appropriate in terms of development, implementation, and project value to the business. The feasibility study concentrates on the following areas:

- Economic feasibility
- Technical feasibility
- Operational feasibility
- Legal feasibility
- Schedule feasibility

Software Engineering MCQ

1.Agile Software Development is based on which of the following type?

- ◯ Incremental Development
- ◯ Iterative Development
- ◯ Both Incremental and Iterative Development
- ◯ Waterfall Development

2.Attributes of good software are

- ◯ Maintainability
- ◯ Functionality
- ◯ Both
- ◯ None of the above

3.The limitation of the spiral model is

- ◯ Not appropriate for small projects
- ◯ Operation cost is high
- ◯ Is not efficient
- ◯ The risk factor is manageable

4.This software development model represents the progress as a linear do